SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: **Apply lexical analysis for token specification and recognition using LEX tool** (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Kola Amrutha Sandhya 192524270 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Amrutha

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

QUESTION 1

Trace the processing of the given C program through the phases of a compiler and identify the errors.

A. Error Identification

This table identifies the errors present in the given C program, their corresponding compiler phase, and the appropriate corrections.

Table 1: Error Identification in Question 1

Line	Error	Correct Classification	Correction
5	i <= n allows one extra array access	Logical error	Use i < n
11	Missing semicolon after array declaration	Syntax error	Add ;
13	sumArray() is called with only one argument	Semantic error	Use sumArray(arr, 5)
15	Missing semicolon after printf()	Syntax error	Add ;
18	Division by zero because b = 0	Runtime error / undefined behavior	Use a non-zero divisor
20	Pointer p is uninitialized before dereference	Runtime error / undefined behavior	Initialize p before *p

This table identifies the errors present in the given C program, their corresponding compiler phase, and the appropriate corrections.

B. Compiler Phases

Source Program
↓
Lexical Analysis
↓
Syntax Analysis
↓
Semantic Analysis
↓
Intermediate Code Generation
↓
Code Optimization
↓
Target Code Generation

1. Lexical Analysis: Converts the source program into tokens such as keywords, identifiers, operators, constants and symbols.

2. Syntax Analysis: Checks whether the tokens follow the grammar of C. Missing semicolons are detected here.

3. Semantic Analysis: Checks meaning and type-related rules, such as the number and type of function arguments.

4. Intermediate Code Generation: Produces an intermediate representation of the program.

5. Code Optimization: Improves the intermediate code without changing the intended result.

6. Target Code Generation: Converts the optimized intermediate representation into target/machine code.

B. Corrected Program

Example: For example, if an array contains 5 elements, the loop should run from index 0 to 4. Using i <= n instead of i < n causes an extra array access. This demonstrates how logical, syntax, semantic, and runtime errors are identified during different compiler phases.

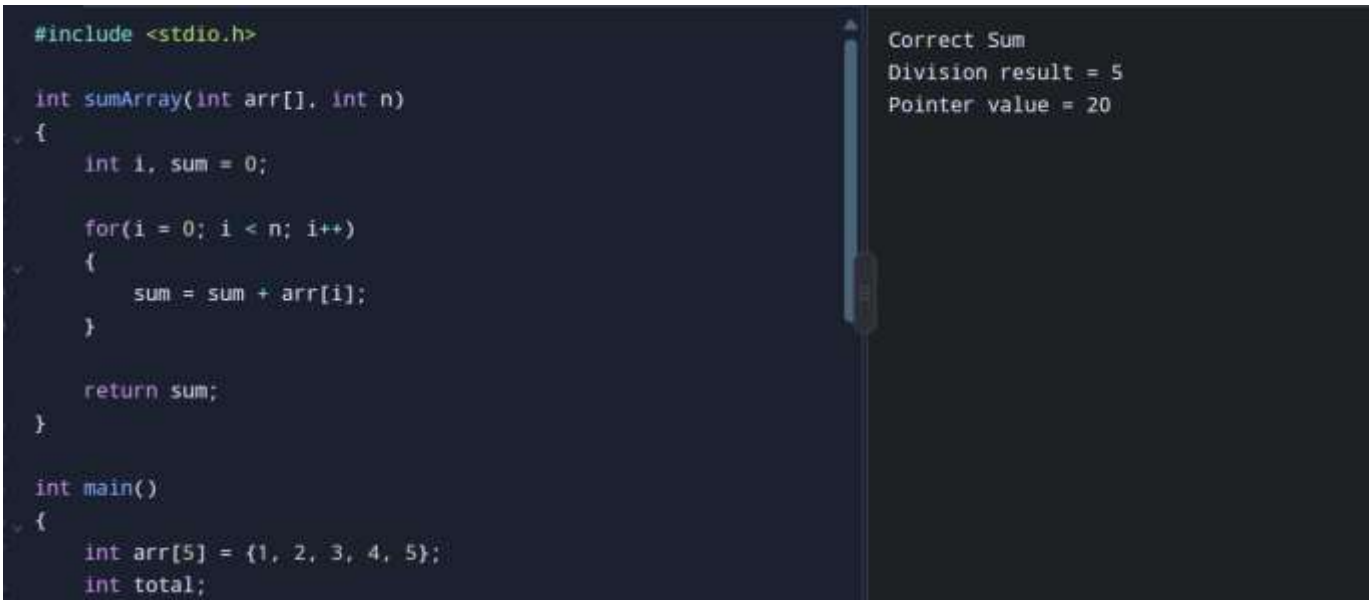
Output:

The below figure shows the sample output obtained after compiling and executing the corrected C program successfully. It demonstrates that the identified errors have been corrected and the program executes as expected.

Division result = 5

Pointer value = 20

Figure 1: Sample Output of the Corrected C Program



QUESTION 2

Identify the invalid tokens and literals in the given C program and explain how the lexical analyzer detects them.

A. Invalid Tokens / Literals

Table 2: Invalid Tokens and Literals in Question

Line	Invalid Construct	Correct Classification	Correction
4	1num	Invalid identifier / tokenization	num1
5	rate@	Invalid character in identifier	rate
6	'abc'	Invalid character constant for intended char use	'a'
7	50$20	Invalid symbol $ in standard C	50 + 20
8	num#1	Invalid character # in identifier	num1
9	%count	Invalid identifier / tokenization	count
11	Unterminated string literal	Lexical error	"Welcome to Compiler Design"
13	09	Invalid integer constant in C (9 is not an octal digit)	9
14	0xZ12	Malformed hexadecimal constant	0x12
15	12.3.4	Malformed floating constant	12.34
17	"Hello	Unterminated string literal	"Hello"
21	invalid-id	Invalid intended identifier; '-' becomes a separate operator token	invalid_id
23	5..6	Malformed floating constant	5.6
24	10@5	Invalid symbol @ in standard C	10 + 5
26	'xy'	Multi-character character constant; not a valid single char value for intended use	'x'
27	3value	Invalid identifier / tokenization	value3

This table lists the invalid tokens and literals found in the program, their classification, and their corrected forms.

C. Identifier Rule

A valid C identifier must begin with a letter (A-Z or a-z) or underscore (_). The remaining characters may be letters, digits, or underscores. An identifier cannot begin with a digit and cannot contain special symbols such as @, #, $, or %.

D. Lexical Analyzer / Scanner

The lexical analyzer scans the source program from left to right and groups characters into tokens. It recognizes keywords, identifiers, constants, operators and separators using predefined patterns. When it encounters an invalid character, malformed number, invalid identifier, or unterminated string, it reports a lexical error and attempts error recovery so that scanning can continue.

E. Important Technical Note

Some constructs in this question are not strictly lexical errors in every C implementation. For example, a multi-character character constant such as 'xy' can be tokenized by a C compiler but may produce a warning or constraint-related diagnostic. Similarly, invalid-id is tokenized as identifier, '-', identifier. The classification above uses the most appropriate compiler-phase classification for assessment purposes.

Example: An identifier such as 1num is invalid because an identifier cannot start with a digit. Similarly, rate@ is invalid because @ is not allowed in a standard C identifier. This demonstrates how lexical analysis identifies invalid tokens and literals.

QUESTION 3

Identify the lexical, syntax, semantic, runtime, logical and optimization-related errors in the given C program. Generate intermediate code and show the optimized form.

Given Program

```
#include <stdio.h>
int divide(int a, int b) {
   return a / b
}
int main() {
   int #value = 20;
   float num = 15.5
   int result;
   result = divide(10);
```

```
    if(result = 5) {
        printf("Result is 5\n")
    }
    int x = 10;
    int y = 0;
    int z = x / y;
    int arr[4] = {1,2,3,4};
    printf("%d\n", arr[4]);
    int *ptr;
    *ptr = 25;
    int total;
    total = x + y * result + 0;

    while(x < 20) {
        printf("%d\n", x);
    }
    return 0;
}
```

A. Error Identification

Table 3: Error Identification in Question 3

Line	Error	Correct Classification	Correction
4	Missing semicolon after return a / b	Syntax error	return a / b;
8	#value is not a valid identifier	Lexical / tokenization error	int value = 20;
9	Missing semicolon after 15.5	Syntax error	float num = 15.5;
12	divide(10) supplies only one argument instead of two	Semantic error	divide(10, 2)
14	Assignment operator used in if condition	Logical / programming error	Use result == 5
15	Missing semicolon after printf()	Syntax error	Add ;
20	y is zero, so x / y causes division by zero	Runtime error / undefined behavior	Use non-zero y
23	arr[4] is outside the valid index range 0 to 3	Runtime error / undefined behavior	Use arr[3]
26	ptr is uninitialized before dereference	Runtime error / undefined behavior	Initialize ptr
29	Adding 0 is redundant	Optimization opportunity	Remove + 0
31-33	x is never changed, so the loop never	Logical error / infinite loop	Increment x using x++

	terminates		

This table classifies the errors into lexical/tokenization, syntax, semantic, runtime, logical, and optimization-related errors and provides the required corrections.

B. Operator Precedence

The expression x + y * result + 0 is interpreted as x + (y * result) + 0 because multiplication has higher precedence than addition. Therefore, the expression itself does not contain an operator-precedence error.

C. Intermediate Code

t1 = y * result
t2 = x + t1
t3 = t2 + 0
total = t3

D. Optimized Intermediate Code

t1 = y * result
t2 = x + t1
total = t2

The optimizer removes the redundant operation + 0. This is an example of algebraic simplification.

E. Other Useful Optimizations

Original Expression	Optimized Expression
x + 0	x
x * 1	x
x * 0	0
x - 0	x

F. Corrected Program

Example: In the expression x + y * result + 0, multiplication is performed before addition because of operator precedence. The unnecessary + 0 can then be removed during optimization. This demonstrates syntax, semantic, runtime, logical, and code-optimization concepts.

Output:

Result is 5

Figure 2: Sample Output of the Corrected C Program

The above figure shows the sample output obtained after compiling and executing the corrected C program successfully. It demonstrates that the identified errors have been corrected and the program executes as expected.

```c
#include <stdio.h>

int divide(int a, int b)
{
    if(b == 0)
    {
        return 0;
    }

    return a / b;
}

int main()
{
    int value = 20;
    float num = 15.5;
    int result;
```

```
Result is 5
4
10
11
12
13
14
15
16
17
18
19
Division result = 5
Total = 20
Pointer value = 25
```